

电机/编码轮记步与参数同步改动整理

本文按功能模块整理当前工作区已经发生的代码改动，不再按日期、追加时间或单次操作拆分。文档重点覆盖电机步进记步、编码轮记步、参数同步、测量流程、CPU3 菜单权限、中文打印和配套资料。

0. 总体说明

这批改动可以归纳为五条主线：

1. 支持编码轮和 TMC5130 `XACTUAL` 电机步进两种记步来源，并通过 `position_count_mode` 参数控制。
2. 增强电机位置可靠性，包括 `XACTUAL/XTARGET` 持久化、运行期轮询、局部首圈周长标定和 TFIT 尺带参数拟合。
3. 调整测量流程，长期跟随类流程进入跟随前切换到电机步进记步，提零点时按当前记步源决定是否检查零点差距过大。
4. 同步 CPU2/CPU3 参数结构、保持寄存器、菜单项、写权限和批量同步边界，避免 CPU3 旧缓存覆盖 CPU2 自动更新字段。
5. 整理串口打印、故障码表和配套文档，使现场调试信息以中文为主，保留必要协议名和寄存器名。

1. CPU2: 设备参数结构与参数存储

涉及文件：

- `LTD_MAIN_CPU2/Services/ParamStorage/system_parameter.h`
- `LTD_MAIN_CPU2/Services/ParamStorage/system_parameter.c`

1.1 复用 reserved5/reserved6/reserved7 为实际参数

`DeviceParameters` 中原来的预留字段已改为电机和记步相关参数：

```
uint32_t motor_current;
uint32_t position_count_mode;
uint32_t motor_count_first_loop_circumference_mm;
```

含义：

- `motor_current`
 - TMC5130 运行电流 `IRUN` 编码值。
 - 合法范围是 `1 ~ 31`，异常值恢复为默认值 `16`。
- `position_count_mode`
 - `0`：编码轮记步。
 - `1`：TMC5130 电机步进记步。
- `motor_count_first_loop_circumference_mm`
 - 电机记步模式使用的当前位置局部首圈周长。
 - 实际存储单位是 `0.001mm`。
 - 例如 `300900` 表示 `300.900mm`。

新增宏：

```
#define POSITION_COUNT_MODE_ENCODER 0u
#define POSITION_COUNT_MODE_MOTOR 1u
#define MOTOR_CURRENT_DEFAULT 16u
#define MOTOR_CURRENT_MIN 1u
#define MOTOR_CURRENT_MAX 31u
```

1.2 新增参数合法性归一化

`system_parameter.c` 新增以下逻辑：

- `MOTOR_LOCAL_CIRC_MIN_001MM = 50000`。
- `MOTOR_LOCAL_CIRC_MAX_001MM = 5000000`。
- `MOTOR_LOCAL_CIRC_FALLBACK_001MM = 600000`。
- `normalize_device_params_runtime()`。
- `device_params_default_motor_local_circ_001mm()`。

加载参数后会执行归一化：

- `position_count_mode` 不是 0/1 时改为编码轮记步。
- `motor_count_first_loop_circumference_mm` 不在 `50.000mm ~ 5000.000mm` 范围时，使用首圈周长换算值。
- 如果首圈周长换算值也非法，使用 `600.000mm`。
- 如果发生归一化，会触发参数写回 FRAM。

1.3 上电成功读取参数后固定打印参数

`init_device_params()` 中，`load_device_params()` 成功后新增：

```
print_device_params();
```

因此上电正常加载参数时也会打印完整 `DeviceParameters`，不再只在保存、恢复出厂或 A/B 分区修复时打印。

1.4 恢复出厂参数新增默认值

`RestoreFactoryParamsConfig()` 中新增默认参数：

- `motor_current = MOTOR_CURRENT_DEFAULT`
- `position_count_mode = POSITION_COUNT_MODE_ENCODER`
- `motor_count_first_loop_circumference_mm = first_loop_circumference_mm * 100`

1.5 参数打印改为中文分组

`print_device_params()` 已改为中文分组输出，电机与编码器部分包含：

- 编码轮周长(0.001mm)

- 电机运行电流(IRUN)
- 电机最大速度(0.01m/min)
- 首圈周长(0.1mm)
- 尺带厚度(0.001mm)
- 记步模式
- 电机局部周长(0.001mm)

2. CPU2: Modbus 参数映射

涉及文件：

- LTD_MAIN_CPU2/Services/Modbus/stateformodbus.h
- LTD_MAIN_CPU2/Services/Modbus/dataanalysis_modbus.c

2.1 保持寄存器替换预留字段

保持寄存器枚举中：

```
HOLDREGISTER_DEVICEPARAM_RESERVED5  
HOLDREGISTER_DEVICEPARAM_RESERVED6  
HOLDREGISTER_DEVICEPARAM_RESERVED7
```

已替换为：

```
HOLDREGISTER_DEVICEPARAM_MOTOR_CURRENT  
HOLDREGISTER_DEVICEPARAM_POSITION_COUNT_MODE  
HOLDREGISTER_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC
```

后续寄存器地址仍按 `REG_STRIDE = 2` 递增。

2.2 参数写入/读出映射新增字段

`WriteDeviceParamsToHoldingRegisters()` 新增写出：

- `g_deviceParams.motor_current`
- `g_deviceParams.position_count_mode`
- `g_deviceParams.motor_count_first_loop_circumference_mm`

`ReadDeviceParamsFromHoldingRegisters()` 新增读回：

- `g_deviceParams.motor_current`
- `g_deviceParams.position_count_mode`
- `g_deviceParams.motor_count_first_loop_circumference_mm`

其中 `motor_current` 读回后会做范围归一化，非法值恢复为默认电流。

2.3 参数读回后同步电机运行态

`ReadDeviceParamsFromHoldingRegisters()` 末尾新增：

```
motorApplyPositionSourceParamsFromDeviceParams();
```

作用是 CPU3 或上位机写入记步模式/电机局部周长后，CPU2 立即同步电机位置源运行态。

3. CPU2: TMC5130 底层驱动

涉及文件：

- `LTD_MAIN_CPU2/Drivers/Peripherals/inc/TMC5130.h`
- `LTD_MAIN_CPU2/Drivers/Peripherals/src/TMC5130.c`

3.1 新增可靠读接口

头文件新增：

```
bool stpr_tryReadInt(TMC5130TypeDef *tmc5130, uint8_t address, int32_t *value);
```

`TMC5130.c` 新增内部函数：

```
static bool tmc5130_tryReadArray(...);
```

实际变化：

- SPI 读寄存器时检查 `HAL_SPI_TransmitReceive()` 返回值。
- 读取失败时返回 `false`。
- 旧 `stpr_readInt()` 保留，但内部调用 `stpr_tryReadInt()`，失败时返回 0 以兼容旧接口。

3.2 修正 32 位寄存器拼接

读回值拼接改成先转 `uint32_t` 再左移：

```
((uint32_t)rxBuff[1] << 24)
```

避免 signed 左移造成未定义行为。

3.3 去掉 SPI 读写中的固定 `HAL_Delay`

`tmc5130_readArray()` 和 `tmc5130_writeArray()` 中原来的固定 `HAL_Delay(10)` 被移除。

3.4 `stpr_moveBy` 增加溢出和读失败保护

`stpr_moveBy()` 从 `void` 改为返回 `uint32_t` 错误码。

新增处理：

- 参数为空返回 `MOTOR_STEP_ERROR`。
- 读取 `XACTUAL` 失败返回 `MOTOR_STEP_ERROR`。
- 使用 `int64_t` 计算 `XACTUAL + ticks`。
- 超出 `int32_t` 范围时打印溢出信息并返回 `MOTOR_STEP_ERROR`。
- 成功后调用 `stpr_moveTo()` 并返回 `NO_ERROR`。

3.5 运动等待增加 SPI 失败退出

`stpr_waitMove()` 修改：

- 读取 `RAMPSTAT` 使用 `stpr_tryReadInt()`。
- 读取失败时调用 `motorQuickStop()` 并返回 `MOTOR_STEP_ERROR`。
- 读取 `GSTAT` 失败时同样快速停止并返回错误。
- 等待循环中调用 `motorPollRuntimePosition()`。
- 运动结束后调用 `motorRefreshDebugDrumState()`。

3.6 stpr_home 增加 SPI 失败退出

`stpr_home()` 等待 `RAMPSTAT.vzero` 时改用 `stpr_tryReadInt()`。

读取失败时：

- 调用 `motorQuickStop()`。
- 直接返回。

3.7 电流设置修正

`stpr_setCurrent()` 的 `IHOLD_IRUN` 写入改为：

```
SET_IHOLD(6) | SET_IRUN(current) | SET_IHOLDDELAY(7)
```

3.8 底层目标/位置变更后触发持久化

以下函数新增 `motorPersistRegistersFromDriver()` 调用：

- `stpr_moveTo()`
- `stpr_moveAngle()`
- `stpr_setPos()`
- `stpr_home()`

4. CPU2: GPIO 初始化

涉及文件：

- `LTD_MAIN_CPU2/Core/Src/gpio.c`

修改内容：

- `MOTOR_SPI2_CS_Pin` 初始化时单独设置为 `GPIO_PIN_SET`。
- `AD5421_SPI3_CS_Pin` 和 `HART_RTS_Pin` 仍初始化为 `GPIO_PIN_RESET`。

目的：TMC5130 的 CSN 低有效，上电空闲应保持高电平。

5. CPU2: 编码器驱动与编码器服务

涉及文件：

- `LTD_MAIN_CPU2/Drivers/Peripherals/src/AS5145.c`
- `LTD_MAIN_CPU2/Services/Encoder/encoder.c`
- `LTD_MAIN_CPU2/Services/Encoder/encoder.h`

5.1 AS5145 错误上报受电机记步模式控制

`AS5145.c` 引入 `motor_ctrl.h`。

以下编码器错误处理增加判断：

- SSI 错误。
- 持续 SSI 错误上报。
- SPI not ready。
- SPI DMA 启动失败。

当 `motorIsPositionSourceMotor()` 为 `true` 时，不再把编码器错误写入全局错误码，也不打印持续错误上报。

5.2 编码器位置更新增加 force 版本

`encoder.c` 将原 `update_sensor_height_from_encoder()` 拆成内部实现：

```
static void update_sensor_height_from_encoder_impl(bool force_position_update)
```

对外提供：

```
void update_sensor_height_from_encoder(void);  
void update_sensor_height_from_encoder_force(void);
```

行为：

- 普通更新在电机记步模式下直接返回，不覆盖主位置。
- force 更新无视位置源模式，仍按编码器刷新位置。

5.3 新增编码器长度独立读取接口

新增：

```
int32_t encoder_get_cable_length_01mm(void);
int32_t encoder_get_sensor_position_01mm(void);
```

这两个函数只根据编码器计数计算值，不写全局 `debug_data`。

6. CPU2: 电机控制核心

涉及文件：

- LTD_MAIN_CPU2/Services/MotorControl/motor_ctrl.h
- LTD_MAIN_CPU2/Services/MotorControl/motor_ctrl.c

6.1 新增对外接口

`motor_ctrl.h` 新增接口包括：

```
void motorRefreshDebugDrumState(void);
void motorPrintPositionRefs(void);
void motorPrintPositionCompare(void);
void motorPollRuntimePosition(void);
void motorApplyPositionSourceParamsFromDeviceParams(void);
void motorResetDrumReferenceToZero(void);
uint32_t motorSwitchPositionSourceToEncoder(void);
uint32_t motorSwitchPositionSourceToMotor(void);
bool motorIsPositionSourceMotor(void);
uint32_t motorCalibrateCurrentTapeCircumference(void);
void motorPersistRegistersFromDriver(void);
void motorTapeFitStart(void);
void motorTapeFitStartLocalOrigin(void);
void motorTapeFitStop(void);
void motorTapeFitAddCurrentSample(void);
void motorTapeFitPrintStatus(void);
uint32_t motorTapeFitSolve(void);
uint32_t motorTapeFitSolveLocalOrigin(void);
uint32_t motorTapeFitApply(bool apply_c0, bool apply_t);
```

6.2 新增电机记步运行基准

`motor_ctrl.c` 新增运行态基准：

```
static int32_t s_motor_count_base_step;
static int32_t s_motor_count_base_length_01mm;
static double s_motor_count_base_turns;
```

用途：

- `base_step`：切换到电机记步瞬间的 `XACTUAL`。

- `base_length_01mm`: 切换瞬间的编码器尺带长度。
- `base_turns`: 由基准长度反推的模型圈数。

电机记步模式下主位置按以下方式更新:

电机源尺带长度 = `base_length` + `XACTUAL` 相对 `base_step` 的变化量

6.3 新增电机局部周长参数编解码

新增函数:

- `Motor_EncodeLocalCircumferenceParam()`
- `Motor_DecodeLocalCircumferenceParam()`
- `Motor_GetLocalCircumferenceFromParams()`
- `Motor_SetLocalCircumferenceToParams()`

这些函数直接读写 `g_deviceParams.motor_count_first_loop_circumference_mm`, 单位为 `0.001mm`。

6.4 新增电机位置源参数应用

`motorApplyPositionSourceParamsFromDeviceParams()` 会:

- 校正非法 `position_count_mode`。
- 校正非法局部周长。
- 当前是电机记步且电机已初始化时, 刷新 `XACTUAL` 并更新主位置。
- 打印当前应用的局部周长。

6.5 新增电机 FRAM 持久化

新增电机持久化记录:

```
typedef struct {
    uint32_t magic;
    uint32_t version;
    int32_t xactual;
    int32_t base_length_01mm;
    int32_t base_step;
    uint32_t crc;
} MotorPersistRecord;
```

保存地址:

- `FRAM_MOTOR_A_ADDRESS` = `FRAM_ANGLE_ADDRESS` + `0x80`
- `FRAM_MOTOR_B_ADDRESS` = `FRAM_MOTOR_A_ADDRESS` + `0x40`

保存内容:

- `XACTUAL`

- 电机记步切换基准长度
- 电机记步切换基准步进

没有保存 `XTARGET`。

6.6 电机持久化恢复逻辑

`Motor_RestorePersistedRegisters()` 会：

- 读取 A/B 双备份。
- 恢复 `XACTUAL`。
- 写 `XTARGET = XACTUAL`。
- 写 `RAMPMODE = HOLD`。
- 恢复 `base_length/base_step` 到运行态暂存。
- 如果 A/B 都无效，保存零位置快照。

6.7 电机持久化保存节流

`Motor_MaybePersistRegisters()` 会：

- 使用 `Motor_TryUpdateDrumState_FromXACTUAL()` 读取可信电机位置。
- 根据当前位置对应卷径计算约 1mm 对应 ticks。
- 位移未达到阈值时不写 FRAM。
- `force=true` 时强制保存。

6.8 防止 XACTUAL 假 0

`Motor_TryUpdateDrumState_FromXACTUAL()` 会：

- 使用 `stpr_tryReadInt()` 读取 `XACTUAL`。
- 如果读到 0，但缓存中的 `motor_step` 非 0，会额外读取 `CHOPCONF`。
- `CHOPCONF` 读取失败或也为 0 时，判定本次 `XACTUAL=0` 无效。
- 无效读数不会刷新电机位置。

6.9 电机调试位置刷新

新增：

- `Motor_SyncDebugDrumState()`
- `motorRefreshDebugDrumState()`
- `motorPollRuntimePosition()`

用途：

- 实时刷新 `debug_data.motor_step`。
- 实时刷新 `debug_data.motor_distance`。
- 电机记步模式下同步刷新 `debug_data.cable_length` 和 `debug_data.sensor_position`。
- 自动 TFIT 采样也在运行轮询中触发。

6.10 位置源切换到编码器

`motorSwitchPositionSourceToEncoder()` 会：

- 设置 `position_count_mode = POSITION_COUNT_MODE_ENCODER`。
- 调用 `update_sensor_height_from_encoder()`。
- 同步电机调试状态。
- 保存位置源参数。
- 打印 `position source switched to encoder`。

6.11 位置源切换到电机

`motorSwitchPositionSourceToMotor()` 会：

- 读取当前可信 `XACTUAL`。
- 使用编码器当前尺带长度作为 `base_length`。
- 使用当前 `XACTUAL` 作为 `base_step`。
- 临时切到电机记步，屏蔽切换窗口内的编码器错误。
- 下行一周。
- 用编码器长度变化计算当前位置局部周长。
- 局部周长合法时写入 `g_deviceParams.motor_count_first_loop_circumference_mm`。
- 上行一周回到原位置。
- 成功后保存参数和电机 FRAM 快照。
- 失败时调用 `Motor_RollbackPositionSourceSwitch()` 回滚旧模式、旧局部周长、旧基准和旧错误码。

6.12 新增当前位置局部周长标定接口

`motorCalibrateCurrentTapeCircumference()` 会根据当前位置、编码器长度和电机步进变化重新计算局部周长，并写入电机局部周长参数。

6.13 新增 TFIT 采样与拟合

新增 TFIT 状态、样本缓存和结果结构。

支持：

- 全局 TFIT：`motorTapeFitStart()`、`motorTapeFitSolve()`。
- 局部 TFIT：`motorTapeFitStartLocalOrigin()`、`motorTapeFitSolveLocalOrigin()`。
- 手动采样：`motorTapeFitAddCurrentSample()`。
- 状态打印：`motorTapeFitPrintStatus()`。
- 参数应用：`motorTapeFitApply()`。

TFIT 采样使用编码器计数换算出的长度，不使用可能被电机记步覆盖的 `debug_data.cable_length`。

6.14 电机初始化恢复顺序调整

`motor_Init()` 首次初始化时新增：

- `Motor_RestorePersistedRegisters()`。
- `Motor_RestorePositionSourceFromParams()`。
- `motorPrintPositionCompare()`。

后续重复调用 `motor_Init()` 不再重新 `stpr_initStepper()`，只重新使能驱动。

6.15 速度计算使用局部周长

以下位置在电机记步模式下优先使用 `motor_count_first_loop_circumference_mm`：

- `Motor_ComputeUniformVelocityFromLength()`。
- `Motor_RefreshVelocityDuringRun()`。
- `motorSetSpeed()`。
- `motorMoveNoWaitWithSpeed()`。
- `motorMoveBlocking_NoDetectWithSpeed()`。

6.16 运动流程增加位置轮询

以下流程增加了 `motorPollRuntimePosition()` 或电机状态刷新：

- `motor_wait_stop_abortable()`。
- `stpr_wait_until_stop_with_target()`。
- `motorMoveAndWaitUntilStopWithSpeed()`。
- `motorMoveUntilCondition()`。
- `motorQuickStop()`。
- `Motor_CheckLostStep_AutoTiming()`。
- `motorMoveBlocking_NoDetectWithSpeed()`。

6.17 电机记步模式下弱化编码器失步报警

`motorMoveAndWaitUntilStopWithSpeed()` 和 `Motor_CheckLostStep_AutoTiming()` 中，如果当前为电机记步模式，检测到编码器位置变化异常时不会按编码器失步错误处理，而是打印当前为电机源并继续。

7. CPU2: 主循环与测量命令

涉及文件：

- `LTD_MAIN_CPU2/Application/Src/app_main.c`
- `LTD_MAIN_CPU2/Application/Src/measure.c`
- `LTD_MAIN_CPU2/Application/Src/measure_zero.c`
- `LTD_MAIN_CPU2/Application/Src/measure_oilLevel.c`
- `LTD_MAIN_CPU2/Application/Src/measure_waterLevel.c`
- `LTD_MAIN_CPU2/Application/Src/measure_tank_height.c`
- `LTD_MAIN_CPU2/Application/Src/test.c`

7.1 主循环增加位置轮询和延迟保存

`App_MainLoop()` 开始处新增：

```
motorPollRuntimePosition();
```

作用是运行期间持续刷新电机位置和电机记步模式下的主位置。

`App_MainLoop()` 末尾新增：

```
process_device_params_deferred_tasks();
```

作用是处理设备参数延迟保存任务，避免设备已经待机时修改参数但没有进入测量流程，导致参数迟迟不写入 FRAM。

7.2 待机全局错误处理中屏蔽编码器错误

`App_HandleIdleGlobalError()` 中新增编码器错误判断。

当当前为电机记步模式，且全局错误码属于编码器类错误时：

- 清除错误码。
- 不进入全局错误状态。

7.3 串口调试协议注释重写

`process_command()` 注释中新增完整调试命令说明，包含：

- A 到 X 的原有调试命令。
- T 系列 TFIT 命令。
- Y 系列位置源切换命令。

7.4 L 命令改为双基准清零

L 命令从直接清 `g_encoder_count` 改为：

```
set_encoder_zero();  
motorResetDrumReferenceToZero();
```

7.5 新增 T 系列 TFIT 串口命令

`process_command()` 新增：

- **T1**：全局 TFIT 开始。
- **T2**：局部 TFIT 开始。
- **T0**：停止采样。
- **TA**：添加当前样本。
- **TS**：打印状态。
- **TR**：全局拟合。
- **TV**：局部拟合。
- **TP**：应用厚度。
- **TU**：应用 C0 和厚度。

7.6 新增 Y 系列位置源命令

`process_command()` 新增：

- **YM**：编码轮切换到电机记步。
- **YE**：切回编码轮记步。
- **YS**：打印位置源和电机/编码轮位置对比。

7.7 液位/水位跟随前加入电机记步切换

`measure.c` 新增 `EnsureMotorPositionSourceBeforeFollow()`。

水位跟随流程变化：

- 先按当前位置源找一次水位。
- 如果当前不是电机记步，则切换到电机记步。
- 切换后重新找水位。
- 再进入水位跟随。

液位跟随流程变化：

- 先处理必要回零。
- 先按当前位置源找一次液位。
- 如果当前不是电机记步，则切换到电机记步。
- 切换后重新找液位。
- 再进入液位跟随。

7.8 零点标定同步清电机基准并切回编码器

`SearchZero()` 成功记录零点后新增：

- `motorResetDrumReferenceToZero()`。
- 流程结束前 `motorSwitchPositionSourceToEncoder()`。

因此零点标定成功后，编码器和电机基准同时归零，并恢复编码轮为主位置源。

7.9 测量日志增加电机/编码参考打印

以下文件中的多处日志新增 `motorPrintPositionRefs()`：

- `measure_oilLevel.c`
- `measure_waterLevel.c`
- `measure_tank_height.c`
- `test.c`

日志会在原传感器位置旁边追加电机/编码轮参考信息。

8. CPU2: Sensor 部件参数读取

涉及文件：

- `LTD_MAIN_CPU2/Services/Sensor/sensor.c`

修改内容：

- `Read_WeightParam_Adapter()` 开始处新增 `motorRefreshDebugDrumState()`。

作用：读取部件参数前刷新电机调试数据，保证返回的 `motor_step/motor_distance` 更新。

9. CPU3: 参数结构、寄存器和菜单

涉及文件：

- `LTD_DISPLAY_CPU3/Application/system_param/system_parameter.h`
- `LTD_DISPLAY_CPU3/Application/system_param/system_parameter.c`
- `LTD_DISPLAY_CPU3/Application/system_param/stateformodbus.h`
- `LTD_DISPLAY_CPU3/Application/display/display_tankopera.h`
- `LTD_DISPLAY_CPU3/Application/display/display_tankopera.c`

9.1 DeviceParameters 新增同名字段

CPU3 的 `DeviceParameters` 中同步使用以下字段：

```
uint32_t motor_current;
uint32_t position_count_mode;
uint32_t motor_count_first_loop_circumference_mm;
```

并新增相同宏：

```
#define POSITION_COUNT_MODE_ENCODER 0u
#define POSITION_COUNT_MODE_MOTOR 1u
#define MOTOR_CURRENT_DEFAULT 16u
#define MOTOR_CURRENT_MIN 1u
#define MOTOR_CURRENT_MAX 31u
```

9.2 CPU3 保持寄存器地址同步修改

CPU3 的保持寄存器枚举中：

- `HOLDREGISTER_DEVICEPARAM_RESERVED5` 改为 `HOLDREGISTER_DEVICEPARAM_MOTOR_CURRENT`。
- `HOLDREGISTER_DEVICEPARAM_RESERVED6` 改为 `HOLDREGISTER_DEVICEPARAM_POSITION_COUNT_MODE`。
- `HOLDREGISTER_DEVICEPARAM_RESERVED7` 改为 `HOLDREGISTER_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC`。

9.3 CPU3 参数表新增菜单项

`param_meta[]` 新增和调整：

- 电机运行电流
 - 操作号：`COM_NUM_DEVICEPARAM_MOTOR_CURRENT`
 - 寄存器：`HOLDREGISTER_DEVICEPARAM_MOTOR_CURRENT`
 - 范围：`1 ~ 31`

- 小数位: 0
- 记步模式
 - 操作号: COM_NUM_DEVICEPARAM_POSITION_COUNT_MODE
 - 寄存器: HOLDREGISTER_DEVICEPARAM_POSITION_COUNT_MODE
 - 范围: 0 ~ 1
 - 小数位: 0
- 电机局部周长
 - 操作号: COM_NUM_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC
 - 寄存器: HOLDREGISTER_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC
 - 范围: 50000 ~ 500000
 - 单位: mm
 - 小数位: 3

9.4 CPU3 菜单分组新增机械参数项

ParamGroupOf() 中将以下操作号归入机械/电机/编码器分组:

- COM_NUM_DEVICEPARAM_MOTOR_CURRENT
- COM_NUM_DEVICEPARAM_POSITION_COUNT_MODE
- COM_NUM_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC

9.5 CPU3 参数打印改为中文分组

CPU3 的 print_device_params() 已改为中文分组输出, 电机与编码器部分包含:

- 电机运行电流(IRUN)
- 记步模式
- 电机局部周长(0.001mm)

10. CPU3: 主板 Modbus 数据映射和参数同步

涉及文件:

- LTD_DISPLAY_CPU3/Communication/internal/main_board_modbus/dataanalysis_modbus.c
- LTD_DISPLAY_CPU3/Communication/internal/main_board_modbus/device_param_sync.c

10.1 CPU3 保持寄存器读写新增字段

WriteDeviceParamsToHoldingRegisters() 和 ReadDeviceParamsFromHoldingRegisters() 中新增:

- motor_current
- position_count_mode
- motor_count_first_loop_circumference_mm

10.2 CPU3 参数同步指针新增字段

get_deviceparam_ptr_by_operanum() 新增:

- COM_NUM_DEVICEPARAM_MOTOR_CURRENT -> &g_deviceParams.motor_current
- COM_NUM_DEVICEPARAM_POSITION_COUNT_MODE -> &g_deviceParams.position_count_mode

- `COM_NUM_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC -> &g_deviceParams.motor_count_first_loop_circumference_mm`

10.3 批量同步跳过 CPU2 自动更新字段

`device_param_sync.c` 新增:

```
static bool DeviceParams_ShouldSkipBulkSync(int operanum)
```

`DeviceParams_SyncAllToCPU2()` 中跳过:

- `COM_NUM_DEVICEPARAM_POSITION_COUNT_MODE`
- `COM_NUM_DEVICEPARAM_MOTOR_COUNT_FIRST_LOOP_CIRC`

原因是这两个字段可能由 CPU2 在 YM 切换和局部周长标定时自动更新, 避免 CPU3 用旧缓存批量覆盖 CPU2 新值。

单项菜单读写仍然保留。

11. CPU3: 参数写权限

涉及文件:

- `LTD_DISPLAY_CPU3/Application/display/display_tankopera.c`

`display_tankopera.c` 新增 `state_allows_param_write(DeviceState state)`, 统一判断当前状态是否允许修改参数。

允许修改:

- `STATE_STANDBY`
- 普通 `0x80xx` 测量完成态

禁止修改:

- `STATE_FLOWOIL`
- `STATE_FOLLOW_WATERING`
- `STATE_SPTTESTING`

这三个状态虽然状态编码符合 `0x80xx` 完成态判断, 但实际属于长期运行态, 不会自动退出, 因此不能按普通完成态放行参数修改。

12. CPU3: 字库与字体检查

涉及文件:

- `LTD_DISPLAY_CPU3/Application/display/display.c`
- `LTD_DISPLAY_CPU3/font_check.py`

12.1 StockMap 补字

StockMap 末尾新增：

记局

12.2 WordStock2 补点阵

WordStock2 新增两个中文字模：

- 记
- 局

用于显示：

- 记步模式
- 电机局部周长

12.3 字体检查脚本同步

font_check.py 中 STOCK_MAP 同步更新，包含新增的 记局。

13. 打印、故障码与资料整理

涉及文件：

- LTD_MAIN_CPU2/Application/Src/fault_manager.c
- LTD_MAIN_CPU2/Application/Inc/fault_manager.h
- LTD_MAIN_CPU2/Services/Sensor/dsm_sensor_communication.c
- LTD_MAIN_CPU2/Services/Sensor/ltd_sensor_communication.c
- LTD_MAIN_CPU2/Services/Sensor/sensor.c
- LTD_MAIN_CPU2/Services/Sensor/wireless_communication.c
- LTD_MAIN_CPU2/Services/Weight/weight.c
- LTD_MAIN_CPU2/Services/Modbus/hostcommu.c
- LTD_DISPLAY_CPU3/Application/app_main.c
- LTD_DISPLAY_CPU3/Application/display/display.c
- LTD_DISPLAY_CPU3/Communication/internal/main_board_modbus/cpu2_communicate.c
- LTD_DISPLAY_CPU3/Application/system_param/cpu3_comm_display_params.c
- LTD_MAIN_CPU2/docs/LTD故障代码表.xlsx
- LTD_MAIN_CPU2/docs/TMC5130A_datasheet_rev1.21.pdf

整理内容：

1. CPU2/CPU3 参数打印改为中文分组格式。
2. 故障打印字段改为中文，命令切换时进入统一错误处理，停止策略更多使用 motorSlowStop()。
3. 液位、水位、罐底、罐高、密度、传感器、称重、Modbus 通讯等日志改为中文描述。
4. 保留必要技术词：CRC、BCC、FRAM、TFIT、XACTUAL、XTARGET、VMAX、COM1/COM2/COM3、UART6 等不翻译。
5. LTD故障代码表.xlsx 按当前 ErrorCode 枚举重新整理，补齐新增电机、编码器、密度测量等故障码，并增加整理说明页。

6. 新增 `TMC5130A_datasheet_rev1.21.pdf`，用于复核 TMC5130 寄存器、状态位、电流配置和位置寄存器行为。

14. 当前运行行为汇总

14.1 什么时候切换到电机步进记步

当前会切换到电机步进记步的路径：

1. 上电参数恢复时，`position_count_mode` 已保存为 `POSITION_COUNT_MODE_MOTOR`，且电机持久化记录有效。
2. 调试命令 `YM` 手动切换到电机步进记步。
3. 液位跟随、水位跟随进入长期跟随前，如果当前仍是编码轮记步，会先切换到电机步进，再重新搜索跟随点。

当前会切回编码轮记步的路径：

1. 调试命令 `YE` 手动切回编码轮记步。
2. 标定零点流程结束后强制切回编码轮记步。
3. 参数归一化发现 `position_count_mode` 非法时，默认恢复为编码轮记步。

14.2 待机时修改参数是否保存

主循环末尾会执行 `process_device_params_deferred_tasks()`，因此设备待机时修改参数也会进入延迟保存任务，不再依赖测量流程退出才能写 FRAM。

14.3 提零点时零点差距报警规则

`measure_zero.c` 新增 `Zero_ShouldCheckDeviation()`：

- 编码轮记步提零点：屏蔽零点差距过大报警。
- 电机步进记步提零点：普通提零点保留零点差距检查。
- 标定零点 `STATE_FINDZEROING`：不做普通零点差距报警，流程结束后切回编码轮记步。
- 零点设置成功后，同时清编码轮零点和电机卷筒参考点。

14.4 CPU3 什么状态允许写参数

CPU3 现在通过 `state_allows_param_write()` 判断参数写权限：

- 待机态允许写。
- 普通测量完成态允许写。
- 液位跟随、水位跟随、固定点监测等长期运行态不允许写。

14.5 调试命令

位置源命令：

- `YM`：切换到电机步进记步，以下行一圈估算当前局部周长，再返回切换前位置。
- `YE`：切回编码轮记步。
- `YS`：打印记步模式、局部周长、`XACTUAL`、编码轮位置、电机位置和差值。

TFIT 命令：

- **T1**: 全局自动采样开始。
- **T2**: 以当前位置为局部原点采样。
- **T0**: 停止采样。
- **TA**: 手动采样。
- **TS**: 打印采样状态。
- **TR**: 全局求解。
- **TV**: 局部求解。
- **TP**: 应用尺带厚度。
- **TU**: 应用首圈周长和尺带厚度。

15. 重点复核风险

1. **XACTUAL/XTARGET** 上电恢复：需要实机确认恢复寄存器不会触发非预期运动，尤其是首次运动前的 HOLD 和目标位置状态。
2. **YM** 切换失败回滚：需要覆盖编码器异常、TMC 通讯异常、运动超时、命令切换中断等场景。
3. 局部首圈周长单位：**motor_count_first_loop_circumference_mm** 使用 **0.001mm**，原 **first_loop_circumference_mm** 使用 **0.1mm**，二者不能混用。
4. CPU3 批量同步跳过字段：如果后续再增加 CPU2 会自动更新的参数，也应加入跳过清单，避免旧缓存覆盖新值。
5. 长期状态写权限：后续新增长期运行态时，需要同步更新 **state_allows_param_write()**。
6. 编码器错误屏蔽边界：只有电机步进作为主位置源时屏蔽编码器通信类错误；TFIT、局部周长标定、切回编码轮仍依赖编码器可信。
7. FRAM 写入频率：电机寄存器持久化按位移阈值和强制保存触发，长期跟随时需要关注写入频率。
8. 慢停替换快停：停止冲击更小，但故障退出耗时会变化，需要确认安全需求是否允许。
9. 日志中文化：如果现场脚本依赖旧英文日志，需要同步更新匹配规则。

16. 构建验证

代码整理过程中已执行并通过：

- **cmake --build D:\CUBE\build\LTD_MAIN_CPU2**
- **cmake --build D:\CUBE\build\presets\LTD_DISPLAY_CPU3**

文档整理本身没有再次修改 C 源码，也没有重新运行构建。

17. 影响文件清单

CPU2 主控源码：

- LTD_MAIN_CPU2/Application/Inc/fault_manager.h
- LTD_MAIN_CPU2/Application/Src/app_main.c
- LTD_MAIN_CPU2/Application/Src/fault_manager.c
- LTD_MAIN_CPU2/Application/Src/measure.c
- LTD_MAIN_CPU2/Application/Src/measure_density.c
- LTD_MAIN_CPU2/Application/Src/measure_oilLevel.c
- LTD_MAIN_CPU2/Application/Src/measure_tank_height.c
- LTD_MAIN_CPU2/Application/Src/measure_waterLevel.c
- LTD_MAIN_CPU2/Application/Src/measure_zero.c
- LTD_MAIN_CPU2/Application/Src/test.c

- LTD_MAIN_CPU2/Application/Src/wartsila_density_measurement.c
- LTD_MAIN_CPU2/Core/Src/gpio.c
- LTD_MAIN_CPU2/Drivers/Peripherals/inc/TMC5130.h
- LTD_MAIN_CPU2/Drivers/Peripherals/src/AS5145.c
- LTD_MAIN_CPU2/Drivers/Peripherals/src/TMC5130.c
- LTD_MAIN_CPU2/Services/Encoder/encoder.c
- LTD_MAIN_CPU2/Services/Encoder/encoder.h
- LTD_MAIN_CPU2/Services/Modbus/dataanalysis_modbus.c
- LTD_MAIN_CPU2/Services/Modbus/hostcommu.c
- LTD_MAIN_CPU2/Services/Modbus/stateformodbus.h
- LTD_MAIN_CPU2/Services/MotorControl/motor_ctrl.c
- LTD_MAIN_CPU2/Services/MotorControl/motor_ctrl.h
- LTD_MAIN_CPU2/Services/ParamStorage/system_parameter.c
- LTD_MAIN_CPU2/Services/ParamStorage/system_parameter.h
- LTD_MAIN_CPU2/Services/Sensor/dsm_sensor_communication.c
- LTD_MAIN_CPU2/Services/Sensor/ltd_sensor_communication.c
- LTD_MAIN_CPU2/Services/Sensor/sensor.c
- LTD_MAIN_CPU2/Services/Sensor/wireless_communication.c
- LTD_MAIN_CPU2/Services/Weight/weight.c

CPU3 显示端源码：

- LTD_DISPLAY_CPU3/Application/app_main.c
- LTD_DISPLAY_CPU3/Application/display/display.c
- LTD_DISPLAY_CPU3/Application/display/display_tankopera.c
- LTD_DISPLAY_CPU3/Application/display/display_tankopera.h
- LTD_DISPLAY_CPU3/Application/system_param/cpu3_comm_display_params.c
- LTD_DISPLAY_CPU3/Application/system_param/stateformodbus.h
- LTD_DISPLAY_CPU3/Application/system_param/system_parameter.c
- LTD_DISPLAY_CPU3/Application/system_param/system_parameter.h
- LTD_DISPLAY_CPU3/Communication/internal/main_board_modbus/cpu2_communicate.c
- LTD_DISPLAY_CPU3/Communication/internal/main_board_modbus/dataanalysis_modbus.c
- LTD_DISPLAY_CPU3/Communication/internal/main_board_modbus/device_param_sync.c
- LTD_DISPLAY_CPU3/font_check.py

文档和资料：

- LTD_MAIN_CPU2/docs/LTD故障代码表.xlsx
- LTD_MAIN_CPU2/docs/TMC5130A_datasheet_rev1.21.pdf
- LTD_MAIN_CPU2/docs/motor_encoder_change_review.md